3.3. Các mô hình học máy đối chứng

Các mô hình học máy đối chứng được xây dựng nhằm đánh giá hiệu quả của phương pháp đề xuất Char-CNN trong bài toán phát hiện URL phishing. Các mô hình này được huấn luyện trên cùng một tập dữ liệu đã được tiền xử lý và trích xuất đặc trưng, đảm bảo tính công bằng trong quá trình so sánh. Dữ liệu đầu vào bao gồm các đặc trưng số phản ánh đặc điểm của URL như độ dài chuỗi, số lượng ký tự đặc biệt, entropy, cũng như sự xuất hiện của các từ khóa nhạy cảm. Những đặc trưng này giúp mô hình học được các dấu hiệu đặc trưng của URL phishing, vốn thường có cấu trúc bất thường hoặc cố tình gây nhầm lẫn.

3.3.1. Decision Tree

3.3.1.1. Khái niệm

Decision Tree là một thuật toán học có giám sát, sử dụng cấu trúc dạng cây để thực hiện phân loại dữ liệu. Mỗi nút trong cây đại diện cho một điều kiện kiểm tra trên thuộc tính, mỗi nhánh là kết quả của điều kiện, và mỗi lá là nhãn dự đoán. Mô hình hoạt động bằng cách chia dữ liệu thành các nhóm nhỏ hơn sao cho các nhóm này càng "thuần nhất" càng tốt.

3.3.1.2. Quy trình thực hiện

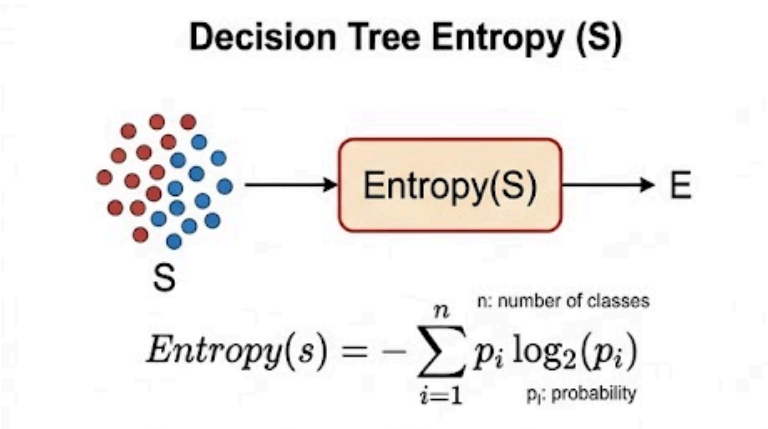
Bước 1: Tiền xử lý và biểu diễn dữ liệu.

Các URL được chuyển đổi từ dạng chuỗi sang dạng vector đặc trưng số thông qua quá trình trích xuất đặc trưng. Việc này là cần thiết vì mô hình Decision Tree không thể xử lý trực tiếp dữ liệu dạng văn bản. Các đặc trưng được sử dụng bao gồm độ dài URL, số lượng ký tự đặc biệt, entropy của chuỗi và sự xuất hiện của các từ khóa nhạy cảm. Những đặc trưng này giúp phản ánh các dấu hiệu đặc trưng của URL phishing trong thực tế.

Bước 2: Tính độ hỗn loạn của dữ liệu bằng Entropy.

Sau khi dữ liệu được chuẩn bị, mô hình tiến hành tính toán độ hỗn loạn của tập dữ liệu tại mỗi nút thông qua chỉ số Entropy, được xác định theo công thức:

$$\text{Entropy}(S) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$



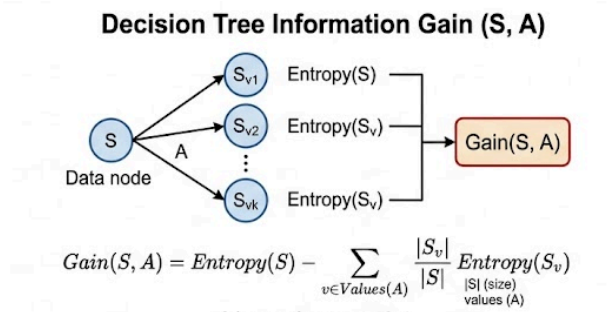
Hình. Công thức dạng hình của Decision Tree (S)

Trong đó, p_i là xác suất của lớp thứ i trong tập dữ liệu. Giá trị Entropy càng thấp cho thấy dữ liệu càng thuần nhất, tức là các mẫu trong tập có xu hướng thuộc cùng một lớp.

Bước 3: Lựa chọn thuộc tính phân chia bằng Information Gain.

Dựa trên giá trị Entropy, mô hình tiếp tục tính toán Information Gain cho từng đặc trưng nhằm xác định thuộc tính tốt nhất để phân chia dữ liệu. Information Gain được tính bằng sự chênh lệch giữa Entropy ban đầu và Entropy sau khi chia, theo công thức:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$



Hình. Công thức dạng hình của Decision Tree (S, A)

Thuộc tính có giá trị Information Gain lớn nhất sẽ được chọn để phân chia, vì nó giúp giảm độ hỗn loạn của dữ liệu một cách hiệu quả nhất.

Bước 4: Xây dựng cây quyết định.
Quá trình phân chia dữ liệu được thực hiện lặp lại theo cách đệ quy. Tại mỗi nút, mô hình tiếp tục chọn thuộc tính tối ưu để chia cho đến khi đạt điều kiện dừng, chẳng hạn như độ sâu tối đa của cây hoặc khi dữ liệu tại nút đã đủ thuần. Trong bài toán phát hiện phishing, quá trình này giúp mô hình học được các quy tắc phân loại như URL dài và chứa nhiều ký tự bất thường thường có khả năng là URL độc hại.

Bước 5: Dự đoán và đánh giá mô hình.
Sau khi hoàn tất quá trình huấn luyện, mô hình được sử dụng để dự đoán nhãn của các URL mới bằng cách đi qua các nhánh của cây từ gốc đến lá. Kết quả thực nghiệm cho thấy Decision Tree đạt độ chính xác tương đối cao, tuy nhiên mô hình có xu hướng overfitting do học quá chi tiết dữ liệu huấn luyện, dẫn đến hiệu suất giảm khi áp dụng trên dữ liệu chưa từng thấy.

3.3.2. Random Forest

3.3.2.1. Khái niệm

Random Forest là một phương pháp học ensemble được xây dựng dựa trên việc kết hợp nhiều mô hình Decision Tree nhằm cải thiện hiệu suất dự đoán và giảm hiện tượng overfitting. Thay vì huấn luyện một cây duy nhất, Random Forest tạo ra nhiều

cây khác nhau bằng cách sử dụng phương pháp lấy mẫu có hoàn lại (bootstrap) từ tập dữ liệu ban đầu. Mỗi cây được huấn luyện trên một tập con dữ liệu khác nhau, đồng thời tại mỗi nút chỉ xét một tập con các đặc trưng, từ đó làm tăng tính đa dạng giữa các cây.

3.3.2.2. Quy trình thực hiện

Bước 1: Tạo tập dữ liệu con bằng Bootstrap Sampling.

Random Forest bắt đầu bằng việc tạo ra nhiều tập dữ liệu con từ tập huấn luyện ban đầu thông qua phương pháp lấy mẫu có hoàn lại. Mỗi tập con này có cùng kích thước với tập ban đầu nhưng có thể chứa các mẫu lặp lại, giúp tăng tính đa dạng của dữ liệu.

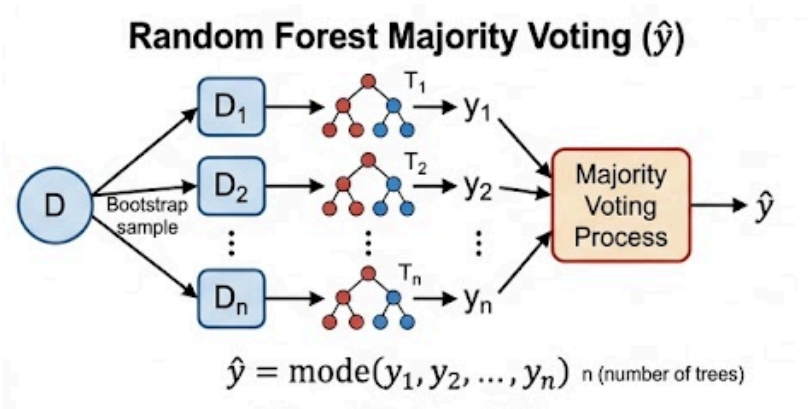
Bước 2: Huấn luyện các cây Decision Tree độc lập.

Trên mỗi tập dữ liệu con, một cây Decision Tree được huấn luyện độc lập. Ngoài ra, tại mỗi nút, chỉ một tập con các đặc trưng được chọn ngẫu nhiên để xét phân chia, giúp giảm sự tương quan giữa các cây và tăng khả năng tổng quát hóa của mô hình.

Bước 3: Tổng hợp kết quả dự đoán bằng phương pháp bỏ phiếu.

Sau khi các cây được huấn luyện, mỗi cây sẽ đưa ra một dự đoán cho đầu vào. Kết quả cuối cùng của mô hình được xác định dựa trên nguyên tắc bỏ phiếu đa số, được biểu diễn như sau:

$$\hat{y} = \text{mode}(y_1, y_2, \ldots, y_n)$$



Hình. Công thức dạng hình của Random Forest

Phương pháp này giúp giảm ảnh hưởng của các dự đoán sai lẻ tẻ từ từng cây riêng lẻ.

Bước 4: Dự đoán và đánh giá mô hình.
Khi áp dụng vào bài toán phát hiện phishing, Random Forest cho thấy khả năng hoạt động ổn định hơn so với Decision Tree. Mô hình đạt hiệu suất cao hơn và giảm hiện tượng overfitting, tuy nhiên vẫn tồn tại một số trường hợp bỏ sót URL phishing, điều này cần được xem xét trong các ứng dụng thực tế.

3.3.3. XGBoost

3.3.3.1. Khái niệm

XGBoost là một thuật toán boosting hiện đại, trong đó các mô hình yếu (thường là cây quyết định) được xây dựng tuần tự, với mỗi mô hình mới được huấn luyện nhằm khắc phục sai sót của các mô hình trước đó. Khác với Random Forest hoạt động song song, XGBoost học theo hướng cộng dồn, liên tục cải thiện dự đoán qua từng vòng lặp.

3.3.3.2. Quy trình thực hiện

Bước 1: Khởi tạo mô hình dự đoán ban đầu.
Quá trình huấn luyện XGBoost bắt đầu bằng một dự đoán ban đầu đơn giản, thường là giá trị trung bình của nhãn trong tập dữ liệu.

Bước 2: Tính toán sai số (Residual).
Sau đó, mô hình tính toán sai số giữa giá trị thực tế và giá trị dự đoán hiện tại theo công thức:

$$\text{residual} = y_{\text{true}} - y_{\text{pred}}$$

Sai số này thể hiện phần thông tin mà mô hình chưa học được.

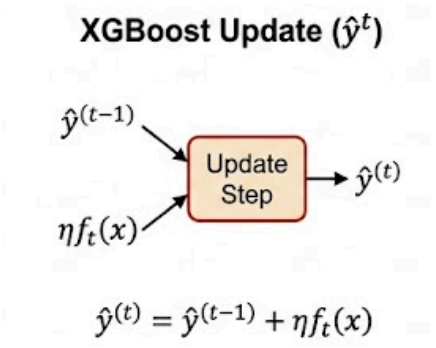
Bước 3: Huấn luyện mô hình mới để học sai số.

Một cây quyết định mới được huấn luyện nhằm dự đoán phần residual này. Nhờ đó, mô hình tập trung nhiều hơn vào các mẫu bị dự đoán sai trong các bước trước.

Bước 4: Cập nhật mô hình dự đoán.

Kết quả dự đoán được cập nhật theo công thức:

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta f_t(x)$$



Hình. Công thức dạng hình của XGBoost

Trong đó, η là tốc độ học giúp kiểm soát mức độ đóng góp của mỗi cây vào mô hình tổng thể.

Bước 5: Lặp lại và đánh giá mô hình.

Quá trình này được lặp lại nhiều lần cho đến khi mô hình đạt hiệu suất tối ưu. Trong bài toán phát hiện URL phishing, XGBoost cho thấy khả năng học tốt các mối quan hệ phức tạp và đạt hiệu suất cao nhất trong các mô hình đối chứng.

3.3.4. MiniLM + Logistic Regression

3.3.4.1. Khái niệm

MiniLM kết hợp với Logistic Regression là một phương pháp tiếp cận dựa trên học biểu diễn, trong đó dữ liệu đầu vào không cần trích xuất đặc trưng thủ công. Cụ thể, mỗi URL được đưa vào mô hình MiniLM để chuyển đổi thành một vector embedding

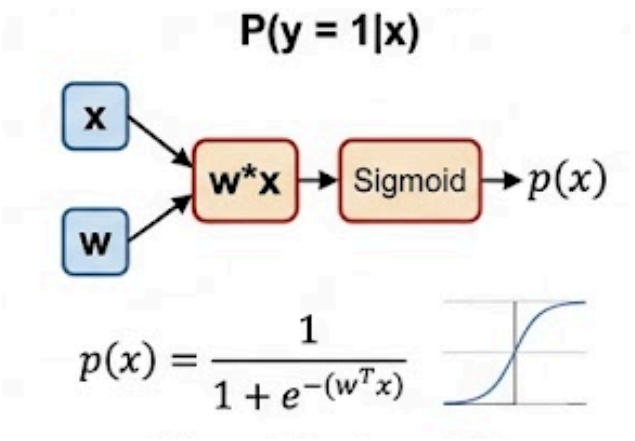
có kích thước cố định, phản ánh ngữ nghĩa của chuỗi ký tự. Vector này sau đó được sử dụng làm đầu vào cho mô hình Logistic Regression để thực hiện phân loại.

3.3.4.2. Quy trình thực hiện

Bước 1: Biểu diễn URL bằng mô hình MiniLM.
Mỗi URL được đưa vào mô hình MiniLM để chuyển đổi thành vector embedding có kích thước cố định. Vector này phản ánh ngữ nghĩa của chuỗi URL mà không cần trích xuất đặc trưng thủ công.

Bước 2: Huấn luyện mô hình Logistic Regression.
Các vector embedding được sử dụng làm đầu vào cho mô hình Logistic Regression để học mối quan hệ giữa đặc trưng và nhãn. Xác suất dự đoán được tính theo hàm sigmoid:

$$P(y = 1|x) = \frac{1}{1 + e^{-w^T x}}$$



Hình. Công thức dạng hình của MiniLM + Logistic Regression

Trong đó, w là vector trọng số và x là vector embedding.

Bước 3: Dự đoán nhãn.
Dựa trên xác suất đầu ra, mô hình phân loại URL thành phishing hoặc hợp lệ bằng cách so sánh với một ngưỡng xác định trước, thường là 0.5.

Bước 4: Đánh giá mô hình.

Kết quả thực nghiệm cho thấy mô hình có khả năng phát hiện URL phishing rất tốt với recall cao, tuy nhiên precision thấp hơn do số lượng false positive lớn. Điều này phản ánh sự đánh đổi giữa việc phát hiện đầy đủ các mối đe dọa và việc giảm thiểu cảnh báo sai.

Kết luận: Tổng thể, các mô hình học máy đối chứng đều đạt hiệu suất cao trong bài toán phát hiện URL phishing, tuy nhiên mỗi mô hình có những ưu điểm và hạn chế riêng. Decision Tree đơn giản và dễ diễn giải nhưng dễ overfitting, trong khi Random Forest cải thiện đáng kể độ ổn định. XGBoost là mô hình mạnh nhất về hiệu suất tổng thể, còn MiniLM + Logistic Regression nổi bật trong việc phát hiện phishing nhờ khả năng học biểu diễn ngữ nghĩa.

Mặc dù vậy, các mô hình này chủ yếu dựa trên đặc trưng được thiết kế thủ công hoặc embedding ở mức từ, chưa khai thác triệt để thông tin ở mức ký tự của URL. Điều này là cơ sở để đề xuất mô hình Char-CNN trong phần tiếp theo, nhằm học trực tiếp từ chuỗi ký tự và nâng cao khả năng phát hiện các mẫu phishing phức tạp hơn.